

QueryNest

PDF RAG Chatbot Using MERN Stack & Local LLM

Project Module & Implementation Blueprint

Brand tagline	Your documents. Your answers.
Product type	AI-powered document knowledge workspace
Core approach	Retrieval-Augmented Generation (RAG) with local LLM inference
Version	Project planning draft · 2026

Branding requirement: The supplied QueryNest logo is the approved logo for this project. Use this logo consistently across the web app, login/landing screens, navigation, documentation, and project presentation. Do not substitute a different logo without an explicit design decision.

1. Project Overview

QueryNest is a web application that lets users upload PDF documents and ask questions in natural language. Instead of answering from general model knowledge alone, the application retrieves relevant passages from the uploaded documents and provides a response grounded in that context. This approach is known as Retrieval-Augmented Generation (RAG).

Problem statement

Reading long PDFs to locate specific facts is slow. QueryNest aims to make document-based information easier to find by letting users ask questions and receive concise answers supported by relevant source excerpts.

Project objectives

- Build a responsive, user-friendly React interface for uploading PDFs and chatting with them.
- Use a Node.js and Express backend for PDF processing, embedding generation, retrieval, and chat orchestration.
- Run the language model locally through Ollama to generate answers from retrieved document context.
- Store document metadata, text chunks, embeddings, and chat history in MongoDB Atlas.
- Show source references or excerpts so users can verify answers against their documents.
- Structure the project into clear modules so it can be tested and extended later.

2. Approved Brand Identity

Product name: **QueryNest**

Tagline: **Your documents. Your answers.**

Logo usage requirement

- Use the exact approved logo shown on the cover as the primary brand logo.
- Keep the logo proportions intact; do not stretch, skew, or recolor it without approval.
- Use the full logo on the landing page, project documentation, and presentation cover.
- Create a compact icon-only variant for the browser favicon or small-screen navigation while retaining the same document-plus-search visual language.
- Maintain strong contrast against the deep navy background. For light surfaces, use a tested alternate version only if needed.

Visual direction

Recommended palette: deep navy (#10131F / #0B2A45), violet (#9A8BFA), and teal (#6BDDD1), with white text and restrained borders. Use a clean sans-serif typeface such as Inter or Manrope. Favor spacious layouts, readable typography, subtle transitions, and clear source citations over decorative effects.

3. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Upload interface, document list, chat UI, history, settings
Routing	React Router	Navigate between app pages
Backend	Node.js + Express	REST API and orchestration
PDF parsing	pdf-parse	Extract text from uploaded PDFs
Upload handling	Multer	Receive uploaded PDF files
LLM runtime	Ollama (local)	Run local chat and embedding models
Chat model	llama3.2:3b (initial choice)	Generate grounded responses; quality and speed depend on hardware/model
Embedding model	embeddinggemma (initial choice)	Convert text and questions into vectors
Database	MongoDB Atlas	Store document metadata, chunks, vectors, and chat history
Semantic search	Atlas Vector Search	Retrieve chunks most relevant to a question

Note: model names and vector dimensions are initial implementation choices. Verify the actual embedding output dimensions and configure the Atlas vector index to match before testing retrieval.

4. System Architecture

- **Step 1 — Upload:** The user selects a PDF in the React interface. The frontend sends it to the Express API.
- **Step 2 — Extract:** The backend validates the upload and extracts text from the PDF.
- **Step 3 — Chunk:** Extracted text is split into overlapping sections so that relevant passages can be retrieved independently.
- **Step 4 — Embed:** Ollama's embedding model converts each text chunk into a numerical vector.
- **Step 5 — Store:** MongoDB Atlas stores document metadata and chunk records, including text and embeddings.
- **Step 6 — Ask:** The user submits a question. The backend embeds the question and searches for the most relevant chunks.
- **Step 7 — Generate:** The backend passes the question and retrieved excerpts to the local chat model through Ollama.
- **Step 8 — Respond:** The UI displays the answer along with source snippets or references to help the user verify it.

High-level flow

React UI	Express API	PDF / RAG pipeline	Ollama + Atlas
Upload & question	Validate & orchestrate	Extract · chunk · retrieve	Embed · search · generate

5. Functional Modules

Module 1 — User Interface & Navigation

Create a responsive application shell with QueryNest branding, a document upload area, document list, chat panel, source citations, chat history, and settings. Provide loading, empty, success, and error states.

Module 2 — PDF Upload & Validation

Accept PDF files only, enforce a configurable file-size limit, use safe server-side filenames, and return clear validation messages. Do not trust client-provided MIME types alone.

Module 3 — PDF Text Extraction

Extract text from supported PDFs. Detect empty or image-only/scanned documents and explain that OCR may be needed for those files. Handle malformed or password-protected PDFs gracefully.

Module 4 — Text Chunking

Split extracted text into manageable overlapping chunks. Start with a tunable chunk size (for example, roughly 800–1,000 words) and overlap, then evaluate answer quality and retrieval accuracy.

Module 5 — Embedding Generation

Generate vectors for each chunk and for each user question using the same embedding model. Validate vector length and handle local Ollama errors or model-not-installed errors.

Module 6 — Document & Vector Storage

Store document metadata and chunk records in MongoDB Atlas. Configure an Atlas Vector Search index for the embedding field and an appropriate filter field such as documentId. Ensure the index dimensions match the chosen model.

Module 7 — Semantic Retrieval

Search for the top relevant chunks for each question. Start with a small result set (for example, four chunks), then tune retrieval count and similarity thresholds based on test questions.

Module 8 — Answer Generation

Send the question and retrieved context to the local chat model. Instruct the model to answer from the provided context, acknowledge when the documents do not contain the answer, and avoid inventing citations.

Module 9 — Chat & Source References

Display the answer and supporting document name/page or chunk excerpt where available. Make citations easy to inspect and preserve clear separation between model-generated text and source material.

Module 10 — Chat History

Save and retrieve conversations and timestamps. For a single-user prototype this can be simple; before multi-user use, scope all records by authenticated user and document permissions.

Module 11 — Error Handling & Observability

Handle missing files, unsupported PDFs, database errors, vector-index errors, Ollama timeouts, and empty retrieval results. Log diagnostic details on the server without exposing secrets or stack traces to end users.

Module 12 — Security & Configuration

Keep MongoDB credentials in environment variables, restrict upload size and file type, validate API inputs, add rate limits where appropriate, and implement authentication/authorization before any public deployment.

6. Suggested Project Structure

```
querynest/
■■■ client/                    # React + Vite frontend
■   ■■■ src/
■       ■■■ components/          # Navbar, UploadBox, ChatMessage, SourceCard
■       ■■■ pages/               # Home, Documents, History, Settings
■       ■■■ services/            # API client
■       ■■■ styles/
■       ■■■ App.jsx
■■■ server/
■   ■■■ routes/                # upload, chat, documents, history
■   ■■■ controllers/           # request/response handling
■   ■■■ services/              # Ollama, PDF parsing, chunking, retrieval
■   ■■■ models/                # MongoDB schemas
■   ■■■ middleware/            # validation, errors, auth (when added)
■   ■■■ config/                # database configuration
■   ■■■ app.js
■   ■■■ server.js
■■■ .gitignore
■■■ README.md
```

7. Development Phases

Phase 1 — Foundation

Create the repository, React/Vite frontend, Express server, environment configuration, MongoDB connection, and branded app shell.

Phase 2 — PDF ingestion

Implement upload validation, PDF text extraction, chunking, and metadata storage. Confirm extracted text is correct before adding AI.

Phase 3 — Local AI

Install Ollama and pull the selected chat and embedding models. Test embedding and chat endpoints independently from the app.

Phase 4 — Vector retrieval

Create and verify the Atlas Vector Search index, store vectors, and test retrieval with questions whose answers are known.

Phase 5 — RAG chat

Connect retrieval to generation, add source excerpts, and test unknown-answer behavior.

Phase 6 — UX polish

Add document management, chat history, loading states, error messages, responsive layouts, and the approved logo.

Phase 7 — QA & deployment

Run functional and security checks, document setup steps, and deploy only after environment and data-access safeguards are in place.

8. Data Model — Initial Proposal

Documents collection: documentId, originalName, fileSize, pageCount (if available), uploadedAt, processingStatus, and optional userId.

Chunks collection: documentId, chunkIndex, text, embedding, pageNumber (if available), and optional userId. Each embedding must be generated by the same model used for query embeddings.

Chats / messages: conversationId, role, content, createdAt, and source chunk references. Add userId and enforce ownership before supporting multiple users.

9. API Endpoints — Proposed

Method	Endpoint	Purpose
POST	/api/documents/upload	Upload and process a PDF
GET	/api/documents	List uploaded documents
DELETE	/api/documents/:id	Delete a document and its associated chunks
POST	/api/chat	Ask a question against a selected document or permitted document set
GET	/api/history	Retrieve conversation history

These are proposed routes, not a claim that they have already been implemented. The final route names can be adjusted during development.

10. Acceptance Criteria

- A user can upload a valid text-based PDF and see a clear processing status.
- The system stores document metadata and searchable chunks successfully.
- A question retrieves relevant passages from the selected document.
- The answer is grounded in retrieved context and includes usable source references where available.
- When relevant evidence is missing, the chatbot says it cannot find the answer in the selected document rather than confidently guessing.
- The interface works on desktop and mobile sizes and uses the approved QueryNest logo.
- Errors from PDF parsing, MongoDB Atlas, Vector Search, or Ollama are handled with useful messages.
- Secrets are not committed to source control, and public deployment is blocked until authentication and document-access controls are implemented.

11. Scope, Limitations & Risks

- **Local inference is not the same as fully local storage.** Ollama runs the model on the developer's computer, but MongoDB Atlas stores data in the cloud. Do not describe the prototype as fully private or 100% local.
- **Scanned PDFs:** PDF text extraction may return little or no text for image-only documents. OCR is a possible future enhancement.
- **Answer quality:** Results depend on PDF quality, chunking, embedding model, retrieval settings, and the chat model. Evaluation with known-answer questions is necessary.
- **Vector index configuration:** Atlas index dimensions and field paths must match the stored vectors and retrieval pipeline.
- **Resource usage:** A 3B model is a practical starting point for 16 GB RAM, but performance varies by operating system, quantization, context length, and other running applications.

- **Security:** Do not expose an unauthenticated endpoint that allows arbitrary document access. Add authentication, ownership checks, limits, and safe deletion before multi-user or public use.

12. Deliverables

- Responsive QueryNest frontend with approved branding.
- Node.js/Express API for upload, processing, retrieval, and chat.
- PDF extraction and chunking pipeline.
- Ollama integration for embeddings and local answer generation.
- MongoDB Atlas document/chunk storage and Vector Search configuration.
- Chat history and source-reference UI, subject to implementation scope.
- README with installation, environment variables, model setup, database/index setup, and run instructions.
- Manual test checklist and known limitations.

13. Initial Setup Checklist

- Install a supported Node.js LTS release and Git.
- Install Ollama and download the chosen chat and embedding models.
- Create a MongoDB Atlas database and configure network access and a least-privilege database user.
- Set local environment variables in an untracked .env file; add .env to .gitignore.
- Verify the embedding model’s output dimensions and create the Atlas Vector Search index accordingly.
- Build and test one step at a time: PDF extraction → chunking → embeddings → retrieval → answer generation → UI.
- Use non-sensitive test PDFs while developing and do not commit uploaded files or secrets to Git.

Project principle
QueryNest should answer from the user's documents, make its sources inspectable, and be honest when the available context does not contain the answer.